

Contents

Research Archive Platform — End-to-End Architecture Overview	1
1. The whole system, at a glance	1
2. Infrastructure (Terraform)	1
3. ECS / Fargate close-up	2
4. ETL pipeline	2
5. Database: loaders → schema	4
6. Test coverage	4
7. Award runtime — how the redesigned UI navigates	7
Where to go next	7

Research Archive Platform — End-to-End Architecture Overview

A single-page tour of the whole system, from Kuali Oracle to the browser. Each section below is one diagram plus the minimum prose to read it; for full technical detail follow the links to [INFRASTRUCTURE.md](#) and [DATABASE_SCHEMA.md](#). Nothing here is new information — it’s the six architecture diagrams built this session, brought together in reading order.

1. The whole system, at a glance

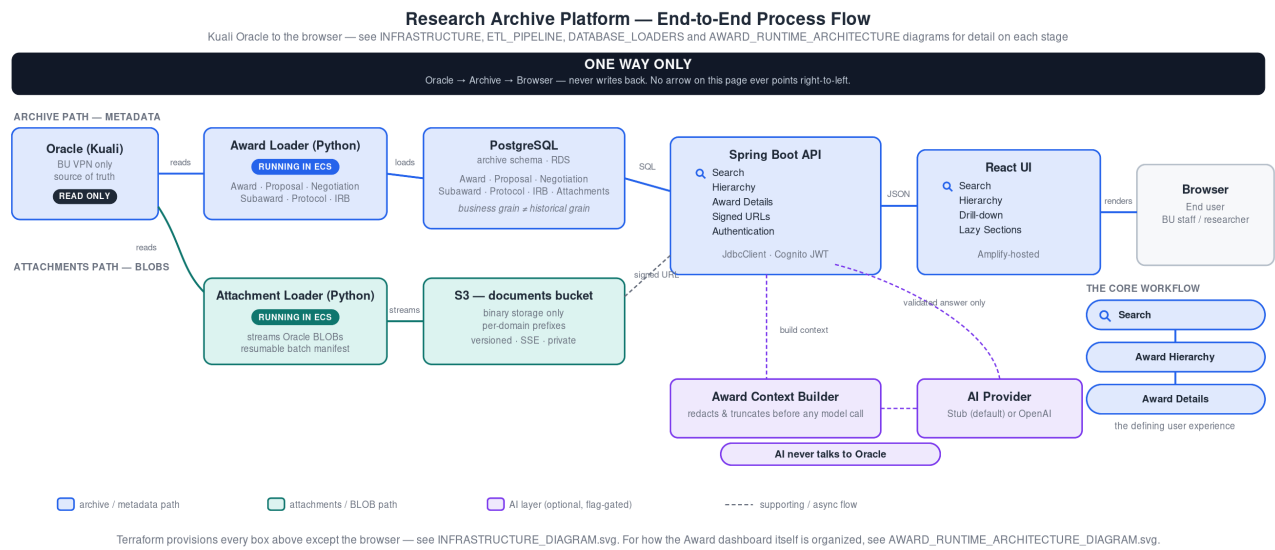


Figure 1: End-to-end process flow

The core path is one-way and read-only: Kuali Oracle → Python ETL → PostgreSQL (archive schema) → Spring Boot API → React UI → browser. Two side paths hang off that spine — document attachments (Oracle BLOB → S3 → signed download through the API) and an optional, flag-gated AI layer that builds a redacted context and validates every model citation before an answer ever reaches the UI. No arrow in this diagram points back toward Oracle; that’s the platform’s central architectural guarantee, not an incidental property.

2. Infrastructure (Terraform)

Every box in §1 except the browser is provisioned by Terraform (terraform/modules/*, wired per environment in environments/{dev, test, prod}/main.tf). The VPC holds public subnets (ALB, optional NAT) and private subnets (both ECS services, RDS), plus interface endpoints so tasks in private

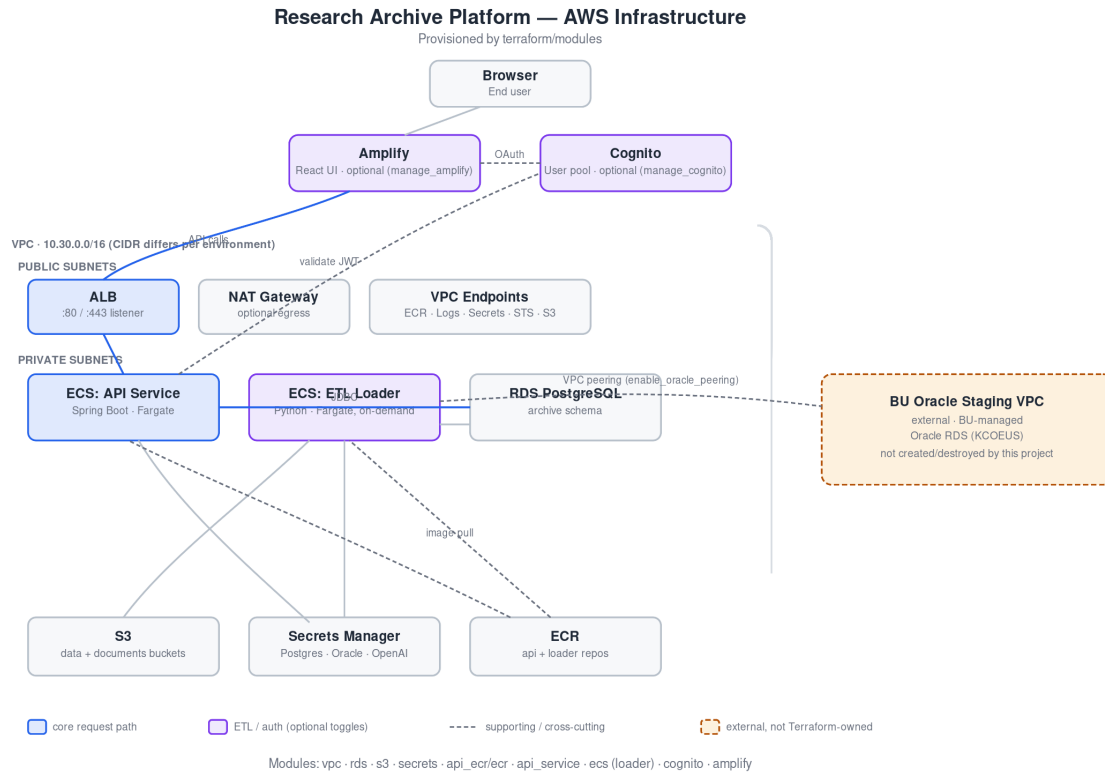


Figure 2: Infrastructure diagram

subnets can reach ECR/Logs/Secrets Manager/STS/S3 without needing NAT for that traffic specifically. Amplify and Cognito are both optional, independently toggled (`manage_amplify`, `manage_cognito`) — either can be “bring your own” while the other is Terraform-managed. The one piece outside this project’s ownership is the BU Oracle staging VPC, reached over VPC peering and represented only as an external, dashed box. Full module-by-module detail, including the dependency graph between modules, is in [INFRASTRUCTURE.md](#).

3. ECS / Fargate close-up

The two Fargate workloads in §2 — API and ETL loader — share an account and VPC but are wired differently on purpose. The API is a long-running `aws_ecs_service` behind the ALB, using ECS-native secret injection: its `execution` role holds `secretsmanager:GetSecretValue` and hands `POSTGRES_*` to the container as plain environment variables before the app starts. The loader has no service at all — it’s launched on demand via `run-task` and exits when done — and deliberately does the opposite: its `execution` role has *no* Secrets Manager access, while its `task` role does, because the loader’s own hardened startup sequence (`identity` → `Postgres` secret → `Oracle` secret → `connectivity` → `migrations` → `table validation`) has to run before any credential resolution happens, and ECS-native injection would defeat that ordering.

4. ETL pipeline

Inside the “ETL (Python)” box from §1: every domain loader follows the same `extract` → `validate` → `transform` → `load` shape (`archive_etl/pipeline/*`, `archive_etl/upload/*`), writing into PostgreSQL inside one transaction per run (`TRUNCATE` + `reload`, fully idempotent). A separate track handles documents — Oracle BLOB streaming through a resumable, deterministic batch manifest (`etl_batch/etl_batch_item`) into the S3 documents bucket. Every loader writes `archive.load_run/load_rejection` audit rows before doing anything risky, and applies pend-

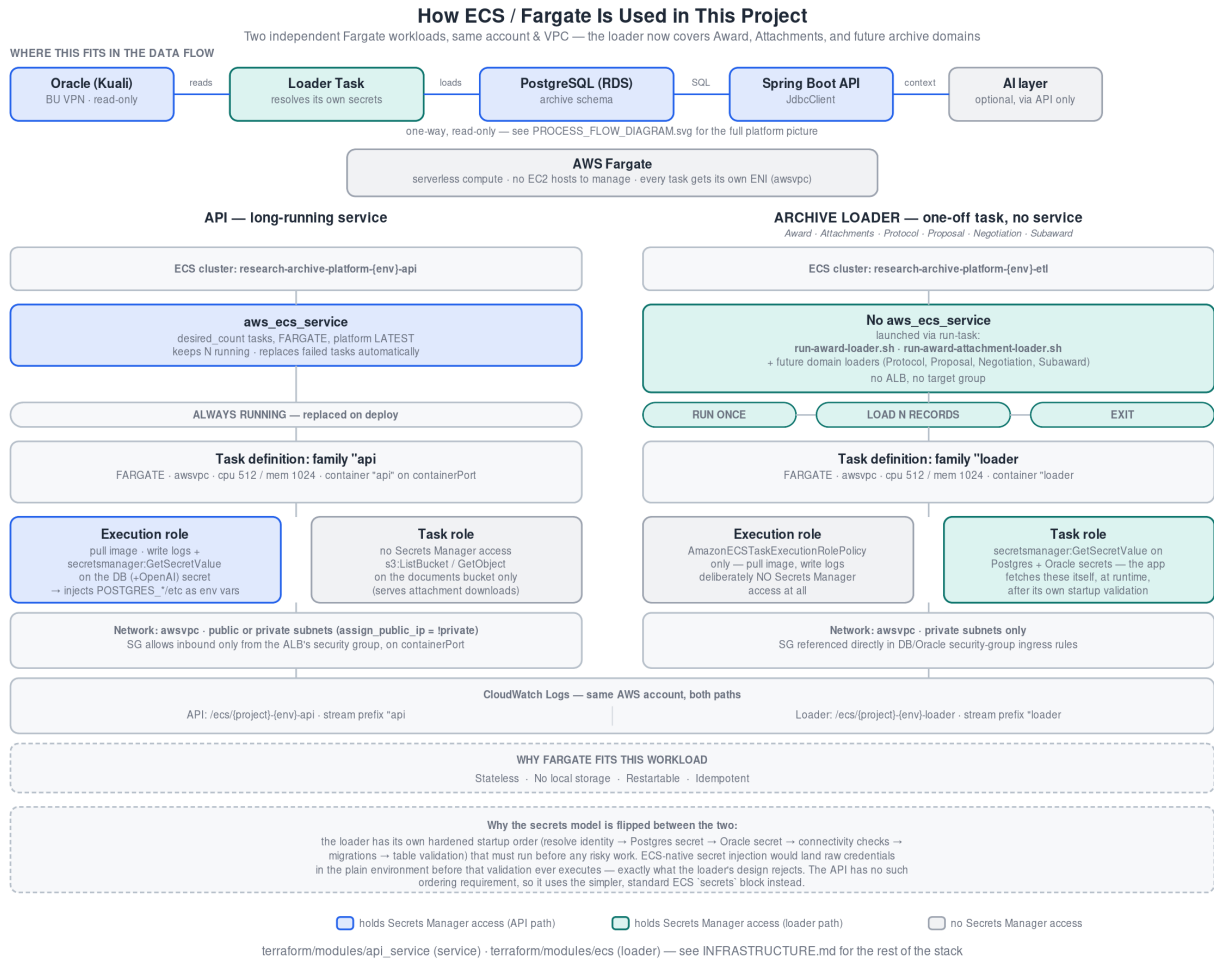


Figure 3: ECS Fargate diagram

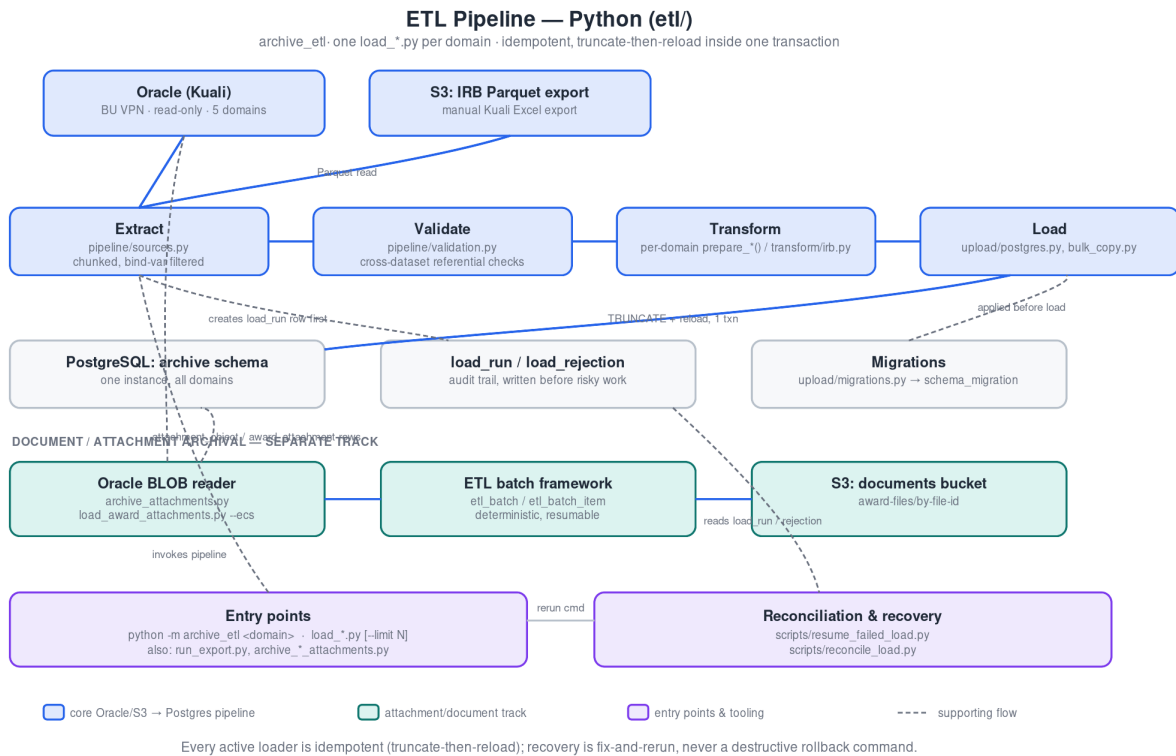


Figure 4: ETL pipeline diagram

ing database/migrations/*.sql itself (Spring Boot’s own Flyway integration is disabled — see [DATABASE_SCHEMA.md](#)). Recovery is always fix-and-rerun; there’s no destructive rollback command by design.

5. Database: loaders → schema

Zooming into the “PostgreSQL” box from §1: seven domains (Award, Proposal, Negotiation, Subaward, Protocol, IRB, Attachments), each owned by one loader script and one parent table plus its children, all living in the same archive schema. Award is the largest by far — about 45 child tables covering amounts, people, budget, terms, compliance, and (as of this session) SAP transmission history — while IRB and Protocol carry their own historical-composite shape distinct from the other domains. Full column-level detail, grain rules (business vs. historical), and the Protocol Archive drop/recreate history are in [DATABASE_SCHEMA.md](#).

6. Test coverage

58 test files back the system: 28 pytest files in etl/tests (mirroring the loader/domain split in §4–5), 28 JUnit files in api/src/test (dominated by the AI feature’s 18 tests — providers, router, citation validator, redactor), and 2 node:test files in the UI covering only AI-presentation-helper formatting (there’s no component-render harness in this repo). The diagram’s one flagged gap, verified against the current source tree: **Award (non-AI), IRB, and Protocol have no dedicated JUnit tests today** — stale references to a ProtocolArchiveControllerTest survive only in api/target/surefire-reports build output, not in source.

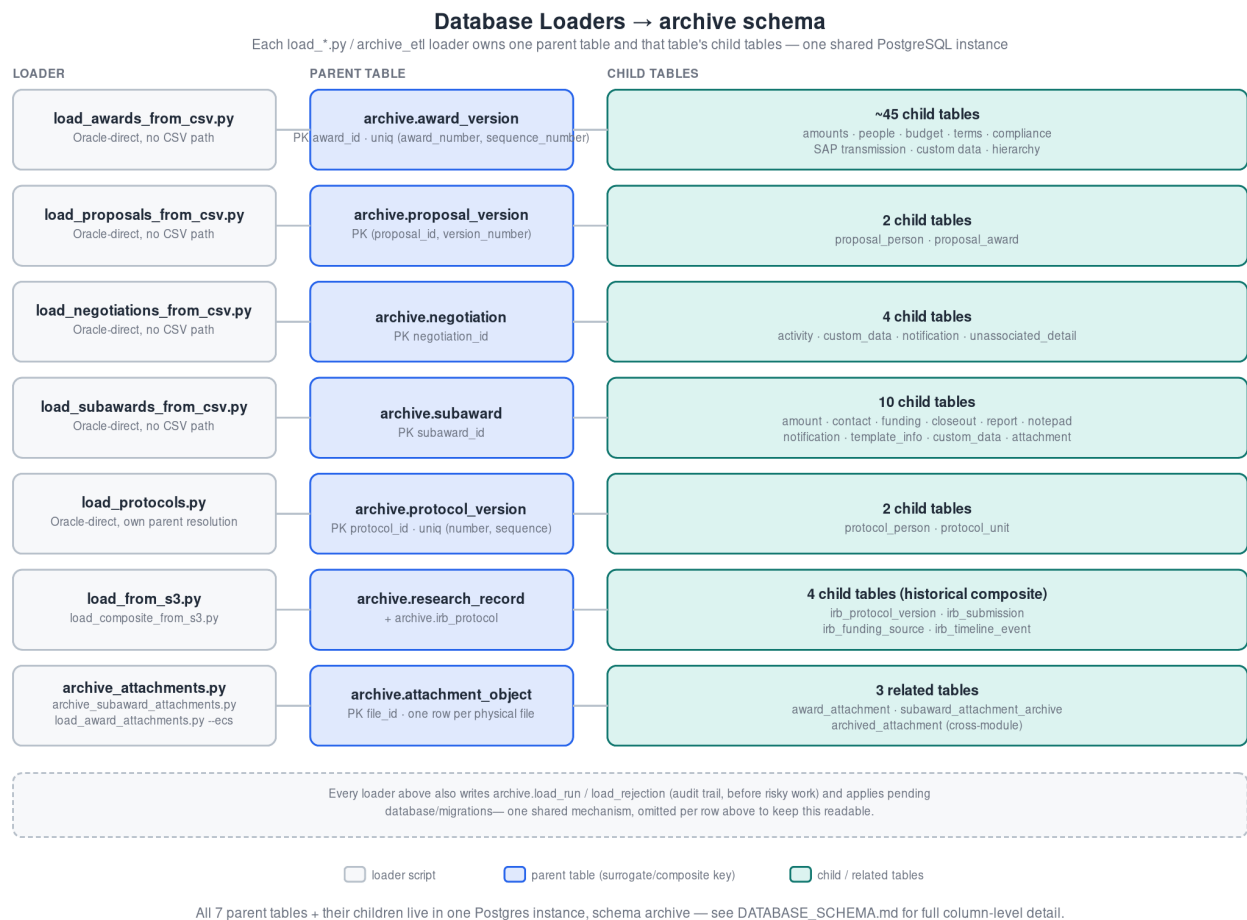


Figure 5: Database loaders diagram

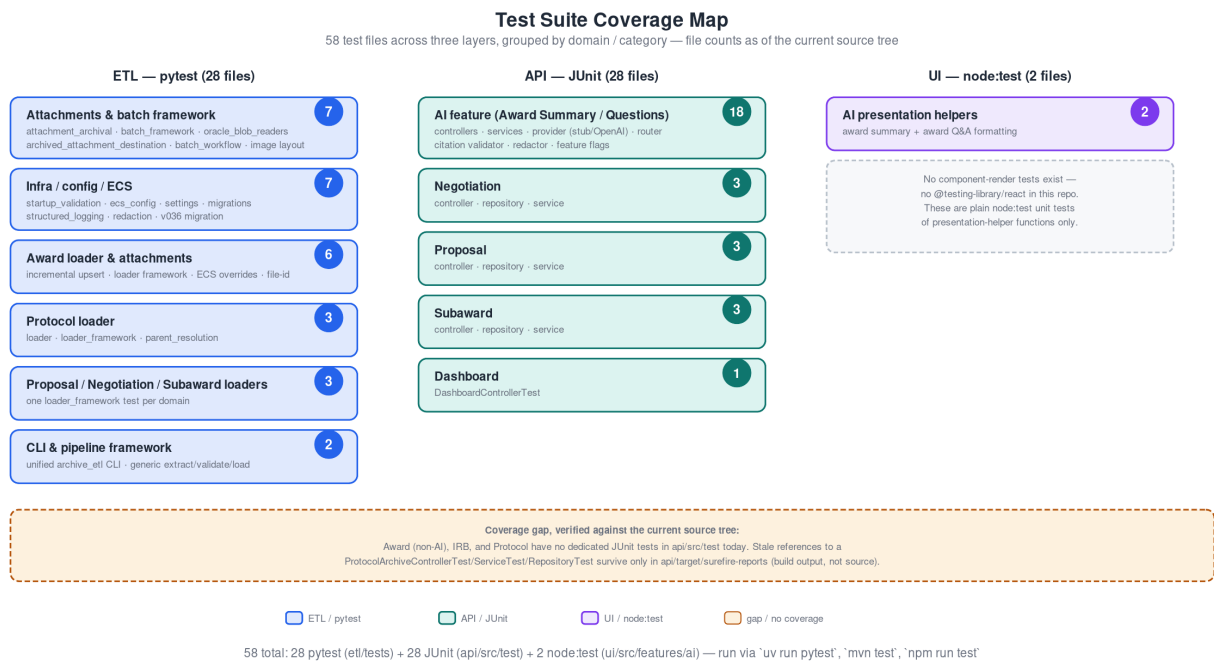


Figure 6: Test coverage diagram

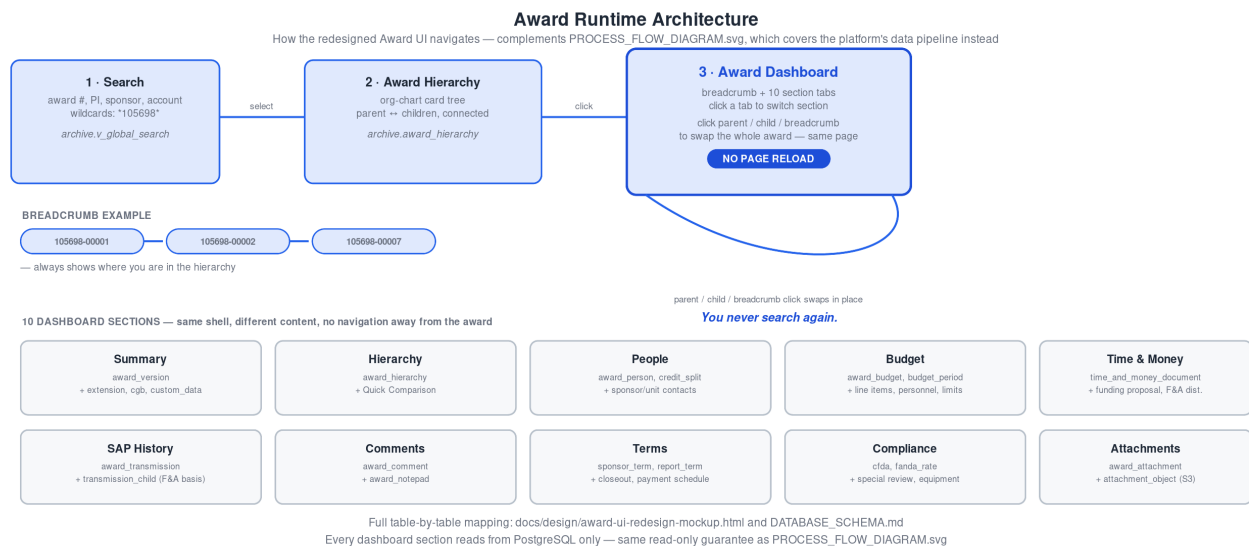


Figure 7: Award runtime architecture

7. Award runtime — how the redesigned UI navigates

Zooming into the “React UI” and “Spring Boot API” boxes from §1, specifically for the Award module’s redesign concept (<docs/design/award-ui-redesign-mockup.html>): three states — Search, a standalone Award Hierarchy tree, and an Award Dashboard with a breadcrumb and ten section tabs (Summary, Hierarchy, People, Budget, Time & Money, SAP History, Comments, Terms, Compliance, Attachments). Clicking any card in the hierarchy or any parent/child/ breadcrumb entry inside the dashboard swaps the whole dashboard to that award in place, with no page reload — the hierarchy *is* the navigation. Every section still only reads from PostgreSQL, so this diagram doesn’t relax the read-only guarantee in §1, it just zooms into one corner of it.

Where to go next

Question	Doc
How do I deploy or change an environment?	terraform/README.md
What does each Terraform module create?	INFRASTRUCTURE.md
What tables exist and how do they relate?	DATABASE_SCHEMA.md
How do I run/debug a loader locally?	etl/README.md
How does the AI feature avoid fabricating facts?	AI_ARCHITECTURE.md
What decisions got reversed, and why?	DECISIONS.md
What does the redesigned Award UI look like/do?	award-ui-redesign-mockup.html